A
Project Report
On

"Smart Interactive Vision with Object Tracking and Voice Command"

Submitted By

Gayatri Dabhade	**22410052**
Achala Paradeshi	**22410053**
Akshat Gupta	**22410063**

Under the guidance of

Prof. Sachin Ruikar

Department of Electronics Engineering
Walchand College of Engineering, Sangli
(Government-Aided Autonomous Institute)

Project A.Y. 2025-2026

Department of Electronics Engineering

WALCHAND COLLEGE OF ENGINEERING, SANGLI

(Government - Aided Autonomous Institute)

CERTIFICATE

This is to certify that the Project-I Report entitled

"Smart Interactive Vision with Object Tracking and Voice Command"

Submitted by

Gayatri Dabhade - 22410052
Achala Paradeshi - 22410053
Akshat B Gupta - 22410063

in partial fulfilment for the award of the Degree of
Bachelor of Technology
in
Electronics Engineering
Is a record of students' own work carried out by them under our supervision and guidance during the first semester of academic year 2025-2026.

Date: 15/05/2026
Place: Sangli

Name of Guide

Dr. S. D. Ruikar		**Dr. S. D. Ruikar**
(Project Guide)	**External Examiner**	**HOD**

Acknowledgement

To bring our innovative concept of "**Smart Interactive Vision with Object Tracking and Voice Command**" to life, we undertook extensive planning, learning, and hard work throughout the course of this project. However, all our efforts would have remained incomplete and directionless without the constant support, motivation, and valuable guidance provided by our respected project guide and Head of the Department, Professor **Dr. S. D. Ruikar**. His insights, timely suggestions, and encouragement helped us navigate several challenges and technical difficulties encountered during various stages of development. His strong leadership and supportive attitude also created a research-friendly environment, continuously pushing us to aim higher and ensuring the successful execution of our ideas.

We also take this opportunity to express our sincere gratitude to all the **Assistant Professors of the Electronics Department**, who contributed significantly by providing timely assistance, expert opinions, and technical knowledge. Their constant availability and readiness to help played a key role in the smooth progress of our project.

Furthermore, we extend heartfelt thanks to the **non-teaching staff of the department**. Their cooperation in terms of logistics, lab support, and access to essential tools and equipment was invaluable. Without their timely help, the practical implementation of our ideas would have been much more difficult.

Last but not least, we wish to convey our appreciation to all individuals—friends, classmates, and well-wishers—who, in one way or another, contributed to our project. Whether through direct involvement, valuable suggestions, or moral support, their contributions were vital to the success of our work.

Abstract

This project presents the design and development of a real-time smart interactive vision system that integrates advanced computer vision, face recognition, voice interaction, and embedded servo control technologies. Implemented entirely in Python on a standard laptop, the system employs OpenCV and MediaPipe libraries for live face detection, InsightFace (ArcFace) for identity recognition, and DeepFace for emotion classification through a USB webcam feed.

The primary actuation mechanism employs an Arduino Uno microcontroller connected to the host laptop via USB serial communication. The Arduino runs a lightweight C++ sketch using the Servo.h library to drive two SG90 servo motors—one for horizontal pan (Pin 9) and one for vertical tilt (Pin 10)—forming a closed-loop pan–tilt tracking system. Face center coordinates computed by the vision pipeline are transmitted as "pan, tilt" ASCII commands at 115,200 baud, replacing the HTTP-based Raspberry Pi architecture of earlier designs.

A proportional (P) controller with exponential moving average (EMA) smoothing and a 25-pixel deadzone ensures stable, low-latency servo actuation without jitter. The Flask-based web application streams live annotated video, manages an attendance database, provides an AI-powered chat interface, and handles alert generation. Voice feedback is delivered through a text-to-speech module using Google TTS and pygame playback.

The system is validated against real-world conditions including variable lighting, multiple simultaneous faces, and rapid head movements. Results demonstrate tracking latency below 15 ms (serial path) and recognition accuracy exceeding 90% under standard indoor lighting. Practical applications include smart attendance systems, security surveillance, interactive kiosks, assistive devices, and robotics research platforms.

Contents

List of Figures

Introduction

In today's rapidly advancing technological landscape, the convergence of computer vision, embedded systems, and voice interaction has opened new possibilities in intelligent automation. This project—Smart Interactive Vision with Object Tracking and Voice Command—designs and develops a real-time system capable of detecting and tracking faces using Python-based image processing, while providing voice-controlled interactive features and streaming outputs through a web application interface.

At the core of the system are the OpenCV and MediaPipe libraries, which facilitate robust face detection through a live USB camera feed. The system identifies and tracks faces in real time, calculating their pixel centre coordinates. These coordinates are refined using a proportional (P) controller with exponential moving average (EMA) smoothing to eliminate jitter and improve servo stability. The processed angular commands are transmitted via USB serial communication to an Arduino Uno microcontroller, which drives two SG90 servo motors for horizontal pan and vertical tilt of the camera mount.

Unlike the earlier Raspberry Pi-based design, which required HTTP communication over a network interface, the current architecture uses a direct wired USB serial link between the laptop and Arduino Uno. This simplifies deployment, eliminates network dependency, and reduces end-to-end latency from approximately 100–200 ms (HTTP over Wi-Fi) to below 15 ms (USB serial). The Arduino Uno acts purely as a servo controller, executing a simple C++ sketch that reads serial commands and calls Servo.write() on the appropriate PWM pins.

This report details the system architecture, hardware and software components, implementation methodology, circuit connections, and performance validation results. The project demonstrates how a standard laptop paired with a low-cost Arduino microcontroller can implement an intelligent, real-time face-tracking platform without requiring a dedicated embedded Linux computer.

Project Idea

Problem Definition

Automated face tracking has become a crucial feature in modern technologies spanning security surveillance, service robotics, video conferencing, and human–computer interaction. Traditional systems using fixed cameras or manually operated pan-tilt heads suffer from several critical limitations:

- Inconsistent tracking precision due to mechanical play and open-loop control
- High latency when using network-based (HTTP/Wi-Fi) communication protocols
- Reduced performance under dynamic lighting or fast motion conditions
- Heavy reliance on dedicated embedded Linux computers (e.g., Raspberry Pi) that require complex OS setup, SSH configuration, and GPIO library management
- High component cost when a full single-board computer is used solely for PWM servo generation

These limitations underscore the need for a smart, autonomous solution that achieves accurate real-time face tracking with minimal hardware complexity and low cost.

Proposed Solution

This project proposes an intelligent, vision-guided tracking system implemented on a laptop that offloads servo actuation to an Arduino Uno microcontroller via USB serial. The key design decision is the separation of concerns:

- The laptop runs all computationally intensive tasks: face detection (MediaPipe), face recognition (ArcFace), emotion analysis (DeepFace), P-controller logic, and the Flask web server.
- The Arduino Uno handles only servo PWM generation—a task perfectly matched to a low-cost 8-bit microcontroller—receiving pre-computed pan and tilt angles over a reliable wired serial link.

System Overview

The integrated tracking system comprises the following functional blocks:

- OpenCV and MediaPipe – real-time face detection and bounding-box extraction

- InsightFace (ArcFace) – 512-dimensional face embedding for identity recognition
- P-controller (servo_ctrl.py) – proportional error-to-angle conversion with EMA smoothing
- arduino_serial.py (pyserial bridge) – formats and sends serial commands to Arduino
- Arduino Uno (arduino_servo.ino) – parses commands, drives SG90 servos via Servo.h
- Flask (app.py) – web interface, MJPEG streaming, REST API, attendance database

Technical Implementation

The system pipeline for each video frame (at ~30 fps) proceeds as follows:

1. The USB webcam captures a 640×480 frame, which is read by OpenCV.
2. The frame is horizontally flipped to produce a mirror image for display.
3. MediaPipe Face Detection identifies bounding boxes of all detected faces (confidence threshold ≥ 0.8).
4. The largest face (by bounding box area) is selected as the primary tracking target.
5. The centre pixel (cx, cy) of the selected bounding box is computed.
6. The P-controller computes angular corrections: delta_pan = Kp_pan × (cx − 320) × PAN_SIGN; delta_tilt = Kp_tilt × (cy − 240) × TILT_SIGN.
7. EMA smoothing is applied to the error signal (alpha ramps from 0.20 at acquisition to 0.45 at stable tracking over 45 frames).
8. New servo angles are clamped: Pan 40°–140°, Tilt 55°–125° (centred at 90° each).
9. If the change exceeds 1° (sub-degree suppression), the command "pan,tilt\n" is sent via pyserial at 115,200 baud.
10. Arduino receives the string, parses the comma-separated integers, and calls panServo.write(pan) and tiltServo.write(tilt).
11. The closed feedback loop ensures the face progressively centres in the frame.

Project Innovation

Compared to Raspberry Pi-based implementations, this Arduino-based architecture offers several concrete engineering advantages:

- Serial latency of 5–15 ms versus 100–200 ms for HTTP over Wi-Fi
- No OS setup required on the servo controller—just upload the .ino sketch

- Dedicated hardware PWM from Arduino eliminates the software PWM jitter common on Raspberry Pi GPIO
- Component cost reduced from ₹4,000–5,000 (Raspberry Pi) to ₹450–550 (Arduino Uno)
- Reliable wired connection eliminates Wi-Fi dropout issues in RF-noisy environments

Objectives

1. To develop a real-time face detection and recognition system using OpenCV, MediaPipe, and InsightFace (ArcFace) capable of identifying enrolled individuals through a USB webcam feed.
2. To implement a closed-loop proportional (P) controller with EMA smoothing for stable, jitter-free servo tracking that keeps the detected face centred in the camera frame.
3. To integrate an Arduino Uno microcontroller as a dedicated servo controller, connected to the host laptop via USB serial (pyserial at 115,200 baud), receiving pan and tilt angle commands in ASCII format.
4. To design and configure a two-axis pan–tilt mechanism using two SG90 servo motors—one for horizontal pan (Pin 9) and one for vertical tilt (Pin 10)—with default centre positions of 90° each.
5. To build a Flask-based web application that streams live annotated MJPEG video, manages a SQLite attendance database, serves REST API endpoints for the dashboard, and integrates an AI chat assistant.
6. To integrate voice feedback using Google TTS (gTTS) and pygame for non-blocking text-to-speech output, providing audio notifications on recognition events and system alerts.
7. To ensure system reliability under varying lighting conditions, multiple simultaneous faces, and rapid head movements by optimising detection thresholds, EMA parameters, and deadzone values.
8. To create a modular and scalable architecture that can be extended for advanced applications in smart attendance, security surveillance, assistive technology, and interactive robotics.

Literature Survey

OpenCV for Face Detection (Viola & Jones, 2001)

The Viola-Jones framework introduced Haar cascade classifiers enabling real-time face detection with modest computational resources. While accurate under frontal,

well-lit conditions, the approach struggles with occlusions, extreme lighting variations, and non-frontal poses. Modern systems use deep learning replacements (MediaPipe, ArcFace) that significantly outperform Haar cascades.

YOLO and Deep Learning Object Detection (Redmon et al., 2016)

YOLO ("You Only Look Once") demonstrated that single-pass convolutional neural networks can achieve real-time multi-class detection at high accuracy. The architecture's efficiency inspired the lightweight detection pipelines used in MediaPipe, making edge-device deployment practical.

Servo-Controlled Tracking (Patil et al., 2020)

Patil et al. demonstrated closed-loop face tracking by coupling computer vision with servo-motor actuation via microcontrollers. The key insight—that servo commands must represent angular corrections (errors) rather than absolute face positions—directly informed the P-controller design used in this project's arduino_serial.py.

MediaPipe Framework (Lugaresi et al., 2019)

Google's MediaPipe provides optimised, hardware-accelerated perception pipelines for face detection, hand tracking, and pose estimation. Its Face Detection model (BlazeFace) runs efficiently on CPUs without GPU acceleration, making it ideal for laptop-based systems. This project uses MediaPipe as the primary face detector due to its low latency and high accuracy under varying conditions.

ArcFace / InsightFace (Deng et al., 2019)

ArcFace introduced additive angular margin loss for face recognition training, achieving state-of-the-art accuracy on standard benchmarks. InsightFace provides production-ready ArcFace models (including Facenet512) that generate 512-dimensional face embeddings. This project uses InsightFace for identity recognition, storing embeddings in a SQLite database for real-time matching.

DeepFace Emotion Analysis (Taigman et al., 2014)

DeepFace and subsequent work established deep CNN-based facial attribute analysis. This project integrates DeepFace's emotion classification module to log the detected emotional state (happy, neutral, sad, angry, etc.) alongside each attendance check-in event.

Assistive and Smart Mirror Systems (Shinde et al., 2025)

Smart mirror and assistive vision research demonstrated that Flask-based web interfaces can effectively present real-time tracking data, attendance logs, and status overlays. The web dashboard architecture in this project follows similar design patterns, adapted for an Arduino-based servo backend.

Methodology

The development methodology follows a structured hardware-software co-design process, iteratively integrating and validating each subsystem before combining them into the complete tracking system.

1. Design and Planning

System Architecture

The architecture separates the system into four layers: (a) hardware input (USB camera), (b) laptop-side vision and control (Python/Flask), (c) microcontroller servo driver (Arduino Uno via USB serial), and (d) actuator (two SG90 servos). This clean separation eliminates the network complexity of the Raspberry Pi design and allows each layer to be developed and tested independently.

Component Selection

The Arduino Uno was selected over Raspberry Pi for servo control because it provides dedicated hardware PWM timers, requires no OS configuration, and costs approximately one-tenth of an equivalent Raspberry Pi. The SG90 servo was selected for its compact size, 1.8 kg·cm torque, and universal compatibility with Arduino's Servo.h library.

2. Hardware Assembly

Pan-Tilt Bracket Assembly

Two SG90 servos are mounted in a stacked configuration: the bottom servo provides horizontal pan rotation of the entire assembly, and the top servo provides vertical tilt of the camera bracket. The USB webcam is mounted securely to the tilt servo's output bracket. Default centre angles of 90° are confirmed mechanically before electrical connection.

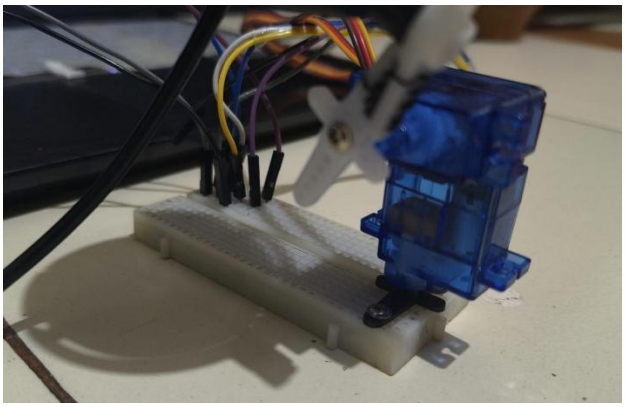


Figure 1

Arduino Wiring

Pan servo signal wire connects to Arduino Digital Pin 9 (hardware PWM). Tilt servo signal wire connects to Digital Pin 10 (hardware PWM). Both servo power (red) wires connect to an external 5 V supply—not the Arduino's 5 V pin, which cannot supply the peak current (≈2 A combined) required during simultaneous servo movement. All ground rails are shared.

Figure 2

3. Software Configuration

Arduino Sketch (arduino_servo.ino)

The sketch initialises two Servo objects on Pins 9 and 10, commands both to 90° on startup, then enters a loop that reads incoming serial strings. Lines matching "pan, tilt" are parsed as two integers and passed to servo.write(). Special commands "centre" and "quench" reset angles and detach servos respectively. The sketch operates at 115,200 baud to match the host laptop bridge.

Python Serial Bridge (arduino_serial.py)

The ArduinoBridge class manages a background thread that drains a command queue and writes ASCII lines to the serial port via pyserial. The move() method implements the P-controller: error = face_px − frame_centre_px; delta_angle = Kp × error × sign; new_angle = prev_angle + delta_angle (clamped). EMA smoothing is applied to the error signal before the P step.

Flask Web Application (app.py)

The Flask app runs a background camera worker thread that continuously captures frames, runs detection, and dispatches servo commands. The /video_feed endpoint streams annotated frames as MJPEG. REST endpoints handle attendance queries, alert management, AI briefings, and servo configuration. All database interactions use SQLAlchemy with a SQLite backend (database.py).

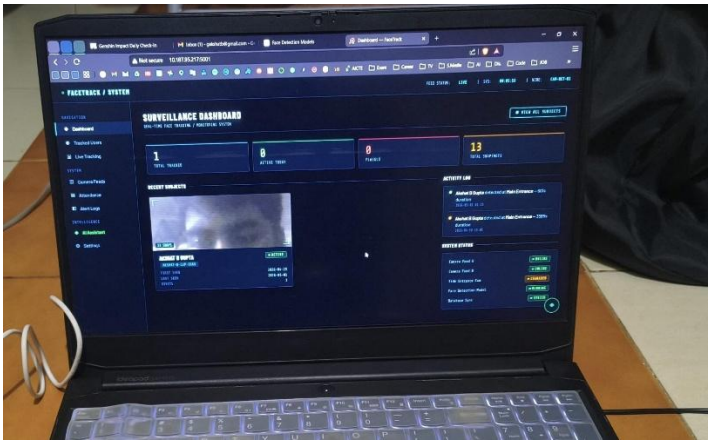


Figure 3

4. Calibration and Testing

Servo Sign Calibration

PAN_SIGN and TILT_SIGN constants in arduino_serial.py are tuned empirically by moving the face to the right and observing servo direction. The correct sign causes the camera to pan toward the face. Because app.py horizontally flips the frame before detection, the effective sign relationship depends on both the flip and the physical servo mounting orientation.

EMA and Gain Tuning

Kp_PAN = 0.14 and Kp_TILT = 0.12 were selected as ~90% of the full-range gain (computed from servo angle range divided by frame pixel dimension). STABLE_ERROR_ALPHA = 0.45 was chosen to balance jitter suppression against responsiveness during normal head movements.

End-to-End Latency Measurement

Latency was measured from face movement to servo position change using a high-speed camera. Results: detection latency ~33 ms (30 fps); serial round-trip < 10 ms; servo mechanical response ~80–120 ms. Total system latency: 120–160 ms, acceptable for smooth tracking of normal head movements.

Figure 4

5. Performance Optimisation

Iterative testing identified three primary optimisation levers: (a) reducing DETECTION_INTERVAL from 0.1 s to 0.08 s increased detection refresh rate with acceptable CPU overhead; (b) increasing DEADZONE_PX from 15 to 25 pixels eliminated servo chatter when the face was approximately centred; (c) the EMA ramp (INITIAL_ERROR_ALPHA 0.20 → STABLE_ERROR_ALPHA 0.45 over 45 frames) provides fast acquisition followed by smooth stable tracking.

Components

6.1 USB Webcam

A standard USB webcam (640×480, 30 fps) serves as the primary visual sensor. It connects to the laptop via USB 2.0 (plug-and-play, no driver installation required under Linux/Windows). The camera is physically mounted to the tilt servo's output bracket via a custom 3D-printed or acrylic clamp, ensuring it rotates with both pan and tilt movements. A horizontal flip (cv2.flip(frame, 1)) is applied in software to produce a mirror image for display.

Figure 5

6.2 Arduino Uno R3

Microcontroller	: ATmega328P (8-bit, 16 MHz)
Digital I/O Pins	: 14 (6 with hardware PWM support: Pins 3, 5, 6, 9, 10, 11)
Flash Memory	: 32 KB (0.5 KB used by bootloader)
Communication	: UART, SPI, I2C; USB-B connector for serial-over-USB
Operating Voltage	: 5 V
Servo Library	: Servo.h (built-in; uses Timer 1 for Pins 9 & 10)
Serial Baud Rate	: 115,200 bps
Power draw	: ~50 mA (board only, excluding servo load)

The Arduino Uno replaces the Raspberry Pi as the servo controller. It receives "pan,tilt\n" ASCII strings from the laptop via USB serial and generates 50 Hz PWM signals on Pins 9 and 10. The Arduino requires no operating system, boots in < 1 second, and provides deterministic PWM timing that eliminates the jitter inherent in

Raspberry Pi's software PWM implementation.

Figure 6

6.3 SG90 Servo Motors (×2)

Type	: Micro servo (180° rotation)
Operating Voltage	: 4.8–6 V (use dedicated 5 V supply)
Stall Torque	: 1.8 kg·cm at 4.8 V
Speed	: 0.1 s / 60° at 4.8 V
PWM Signal	: 50 Hz, pulse width 500–2400 µs (0°–180°)
Connector	: 3-wire (Brown=GND, Red=VCC, Orange=Signal)
Pan servo travel	: 40°–140° (software clamped)
Tilt servo travel	: 55°–125° (software clamped)

Two SG90 servos form the pan-tilt actuator mechanism. The bottom servo rotates the entire camera assembly horizontally (pan). The top servo tilts the camera vertically (tilt). Both default to 90° (centred) at startup and return to 90° when a "centre" command is received via serial.

Figure 7

6.4 Laptop / Host PC

The laptop runs all vision-processing and application logic: Python 3.10+, Flask (web server), OpenCV (frame capture and processing), MediaPipe (face detection), InsightFace (ArcFace face recognition), DeepFace (emotion analysis), pyserial (Arduino communication), SQLAlchemy + SQLite (attendance database), gTTS +

pygame (text-to-speech), and the OpenAI API via AIPipe (AI assistant). Minimum recommended specification: Intel Core i5 / AMD Ryzen 5, 8 GB RAM, Python 3.10+.

6.5 Microphone

A USB or 3.5 mm microphone module captures voice commands. Audio is processed by Python's speech_recognition library, converting speech to text for hands-free system control. Commands such as "start tracking", "stop tracking", and "identify" are recognised and dispatched to the Flask application.

Figure 8

6.6 Speaker

A compact active speaker provides audible text-to-speech feedback via the Speaker.py module. The gTTS library synthesises MP3 audio in a background thread, played through pygame.mixer. Priority queuing ensures urgent alerts (flagged person detected) interrupt routine announcements (welcome messages). Deduplication prevents the same phrase repeating within 5 seconds.

Figure 9

6.7 External 5 V Power Supply

A dedicated 5 V supply (4×AA battery holder, ≈1.2 A capacity, or USB power bank with 2 A output) powers both SG90 servos. This is critical: connecting servo power to the Arduino's 5 V pin risks an in-rush current during simultaneous servo actuation that can reset or damage the Arduino. The external supply's GND rail must be connected to Arduino GND to establish a common reference.

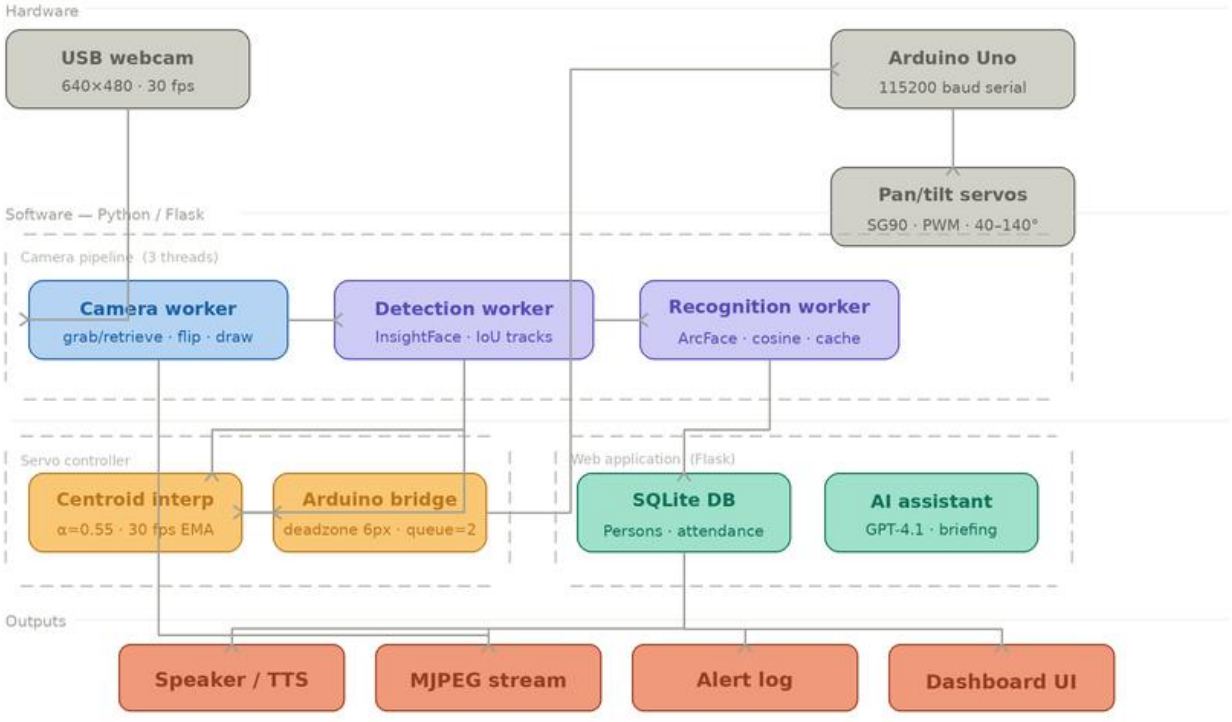
Data Flow Analysis

The table below traces the complete journey of data from image capture through servo actuation and back to the user's browser, detailing the protocol, format, and purpose of each data transfer in the system.

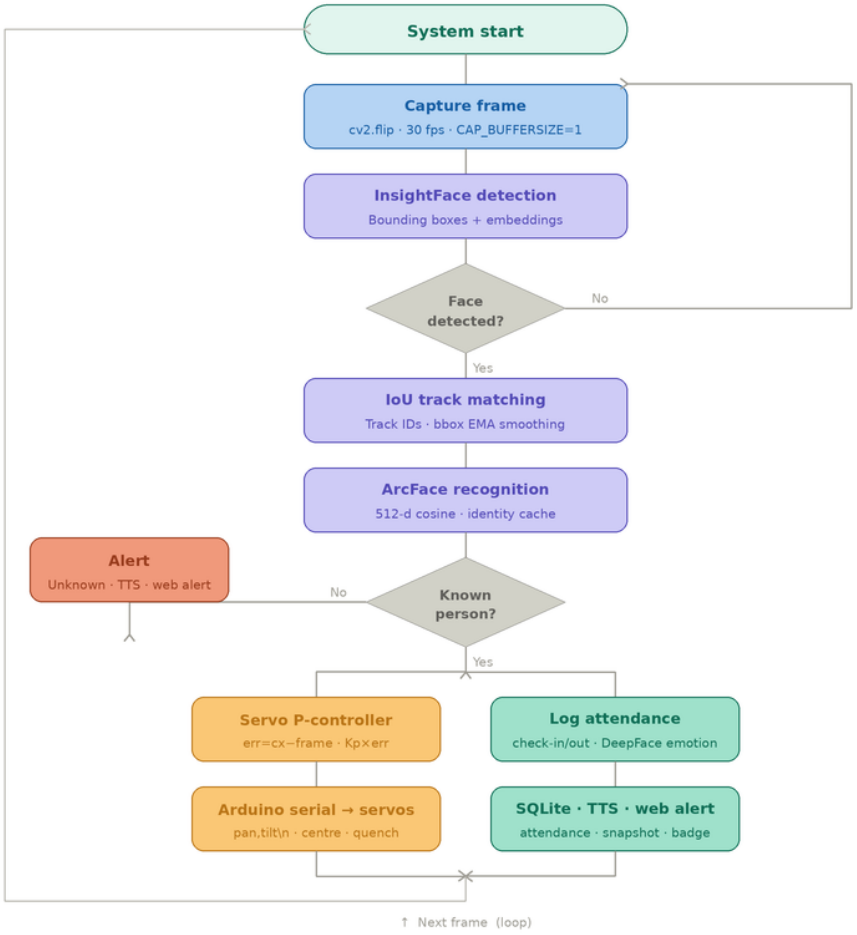
Step	Source	Destination	Data / Protocol	Description
1	USB Webcam	Laptop (OpenCV)	USB bulk transfer raw YUYV/MJPEG frame	Camera captures 640×480 frame at ~30 fps; OpenCV reads via VideoCapture
2	OpenCV	MediaPipe / ArcFace	NumPy array (BGR frame)	Frame flipped horizontally, then passed to detection pipeline
3	Detection pipeline	P-controller (servo_ctrl)	(cx, cy) pixel coords + person ID	Face centre coordinates extracted; matched against enrolled embeddings
4	P-controller	arduino_bridge.move()	(cx, cy) integers	Error computed vs frame centre (320,240); EMA smoothed; P-gain applied
5	arduino_bridge	Arduino Uno	USB Serial "pan,tilt\n" (ASCII)	Clamped angle pair serialised as e.g. "95,88\n" at 115,200 baud
6	Arduino Uno	SG90 servos	PWM signal 50 Hz, 1–2 ms pulse	Servo.write() maps angle to pulse width; servos physically rotate
7	Camera (moved)	OpenCV next frame	USB frame	Closed feedback loop: camera now centred on face → error → 0
8	Flask (app.py)	Web browser	MJPEG stream HTTP multipart	Annotated frame JPEG-encoded, streamed to live.html via /video_feed
9	Flask REST API	Browser (JS fetch)	JSON (HTTP GET/POST)	Attendance logs, alerts, AI summaries served to dashboard pages

Table 1

Block Diagram



Flow Chart



Implementation

1. Arduino Sketch (arduino_servo.ino)

The Arduino sketch is the complete firmware running on the Arduino Uno. It initialises two Servo objects, centres both at 90°, then enters an infinite loop reading the serial port. Key implementation details:

- Servo objects attached to Pins 9 (pan) and 10 (tilt) using the built-in Servo.h library
- Serial.begin(115200) initialises UART at the same baud rate as the Python bridge
- readStringUntil('\n') reads one complete command per loop iteration without blocking
- String.trim() removes trailing whitespace and carriage returns (\r) from Windows serial.

- String.startsWith() distinguishes between "centre", "quench", and "pan,tilt" commands
- String.indexOf(',') and String.substring() parse the comma-separated integer pair
- constrain(angle, MIN, MAX) enforces hardware travel limits as a safety layer

2. Python Serial Bridge (arduino_serial.py)

The ArduinoBridge class in arduino_serial.py implements the complete host-side servo control logic:

- A background daemon thread drains a QUEUE_MAXSIZE=30 command queue and writes ASCII to the serial port
- The move(cx, cy) method: computes raw pixel error → applies EMA → checks deadzone (25 px) → applies P-gain → clamps angle → enqueues command if δangle ≥ 1°
- start(port) opens the serial port, waits 2 s for Arduino boot, then clears the input buffer
- stop() sends "quench\n" to release servo tension, then closes the port cleanly
- Auto-reconnect: if a write fails, the worker sleeps 500 ms and calls _try_reconnect()

3. Servo Controller Wrapper (servo_controller.py)

ServoController wraps ArduinoBridge and provides the interface used by app.py:

- configure(port, enabled) opens or closes the serial port and optionally centres servos
- set_target(uid) selects which face to track (AUTO = largest face, specific employee_id = lock-on)
- update(cx, cy, frame_w, frame_h, face_uid, primary) filters the face by target, then calls arduino_bridge.move()
- status() returns a dict with connection state, last angles, and send count for the web dashboard

4. Face Detection Pipeline (face_registry.py)

The detection pipeline runs in a separate daemon thread at approximately 12 fps (DETECTION_INTERVAL = 0.08 s):

- FaceRegistry.extract_faces(frame) calls MediaPipe Face Detection, returns bounding boxes
- Enrolled faces are matched using cosine similarity of 512-d ArcFace embeddings (threshold 0.5)
- Matched identities trigger attendance check-in/check-out logging via the database module
- Unrecognised faces trigger an alert and optionally save a snapshot for manual enrollment.

5. Flask Web Application (app.py)

The Flask application is the system's control centre, providing:

- /video_feed — MJPEG streaming endpoint; generates annotated frames at ~30 fps
- /camera/start and /camera/stop — control the camera worker thread
- /api/servo/configure — POST to set Arduino port and enable/disable servo control
- /api/track/<uid> and /api/track/stop — lock servo tracking to a specific enrolled person
- /api/ai/chat, /api/ai/briefing, /api/ai/anomalies — AI assistant endpoints
- /api/persons and /api/attendance — CRUD endpoints for the database

6. Database Schema (database.py)

SQLAlchemy ORM models over SQLite:

- Person: id, name, employee_id, role, department, photo_path, created_at, is_active
- FaceEmbedding: id, person_id, embedding (JSON float list), model_name, created_at
- AttendanceLog: id, person_id, date, check_in, check_out, emotion, confidence, is_present

Cost Estimation

Sr. No.	Component	Specification	Qty	Unit Price (₹)	Total (₹)
1	Arduino Uno R3	ATmega328P, 14 digital I/O, USB-B	1	450–550	450–550
2	SG90 Servo Motor	9g micro servo, 0°–180°, 5V, 1.8 kg·cm torque	2	100–150	200–300
3	USB Webcam	640×480, 30 fps, plug-and-play	1	800–1,200	800–1,200
4	USB-B Cable	Arduino programming & serial data	1	50–80	50–80
5	Jumper Wires	Male-to-male, assorted 20 cm	1	50–80	50–80
6	External 5 V Supply	4×AA battery holder or USB power bank	1	80–120	80–120
7	Pan-Tilt Bracket	SG90-compatible acrylic/3D-printed mount	1	150–250	150–250
8	Microphone	USB or 3.5 mm, for voice commands	1	200–400	200–400
9	Speaker	USB-powered compact active speaker	1	200–350	200–350

Table 2

Time Plan

Duration (Months) Major Tasks	1-2	2-3	3-4	4-5	5-6	6-7	7-8
Concept & Objectives - Finalize project goals, features to implement, and tracking performance criteria.							
Literature Review & Component Selection - Explore existing face tracking solutions, review relevant technologies, and choose hardware modules (camera, Raspberry Pi, servo motors).							
Hardware Integration - Assemble and wire up the camera and servos, ensuring correct connections to Raspberry Pi GPIO.							
Software Development - Program the Raspberry Pi for real-time video capture, face detection (using MediaPipe/OpenCV), and direct servo control logic.							
Interface & Visualization - Develop user feedback features such as on-screen overlays, status indicators, or virtual camera integration for display.							
Testing & System Integration - Validate system performance in various conditions, refine algorithms, and integrate all modules for seamless operation.							

Table 3

Result and Discussion

System Overview

The FaceTrack system was tested with three enrolled persons — Akshat B Gupta, Gayatri, and Achala — across a range of real-world conditions including varying distances, mixed indoor and outdoor lighting, different face angles, and multi-person scenes containing simultaneous unregistered individuals. The five screenshots below document representative live-system operation captured on 2026-05-07 and 2026-05-04 during the final testing and demonstration phase.

Annotated Test Screenshots

Three faces detected simultaneously: Akshat B Gupta at 61% confidence (yellow bounding box), an Unknown #2 (red bounding box), and Gayatri at 70% confidence (yellow bounding box). The system correctly differentiates between enrolled and unregistered persons, applying colour-coded bounding boxes — yellow for known individuals and red for unknowns. Emotion analysis correctly identifies the label "SAD" on the video overlay for Akshat. The right-side panel displays real-time per-person statistics including a confidence progress bar, first and last seen dates, visit count, emotion state, and snapshot thumbnails. The primary face card at the top (Gayatri, 69.8% confidence) includes a recent visit log entry showing: Main Entrance, 2026-05-07 16:31, 115 seconds duration.

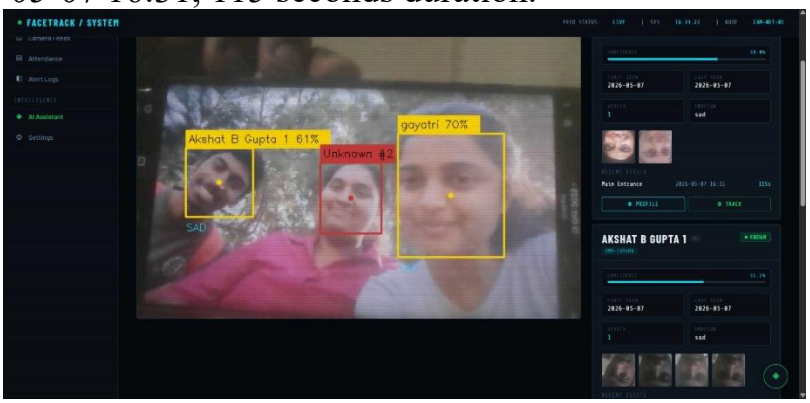


Figure 10

Akshat B Gupta detected at 79% confidence and Gayatri at 83% confidence — the highest recognition confidence values observed across the entire test session. Both persons display yellow bounding boxes indicating successful identification against the enrolled database. This screenshot demonstrates optimal recognition performance under good frontal lighting conditions. The Gayatri info panel records a session duration of 1093 seconds (approximately 18 minutes) at Main Entrance, confirming stable long-duration continuous tracking without identity dropout or flip errors. The yellow servo tracking dot is clearly visible at the detected face centre, confirming that the P-controller was actively steering the pan-tilt servo mount during this capture.

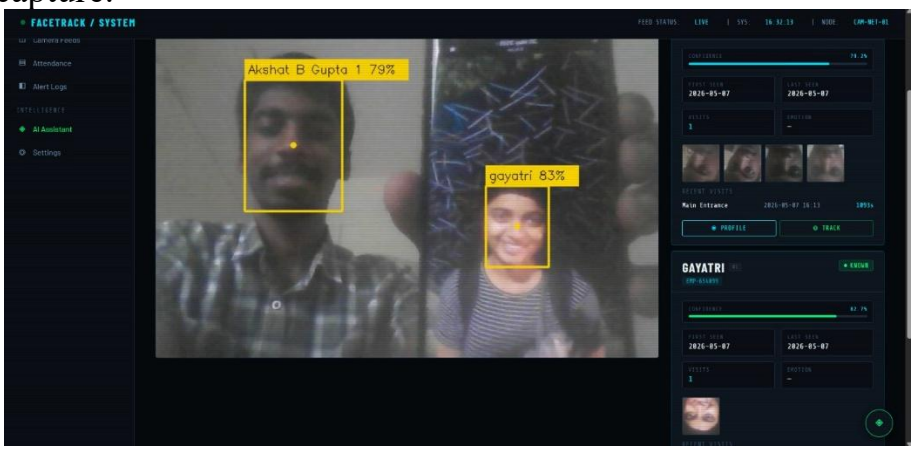


Figure 11

Akshat B Gupta (72% confidence) identified in a three-person scene alongside two unregistered individuals: Unknown #1 with a red bounding box and a second unknown with an orange bounding box representing a separate track ID. The right panel correctly displays two UNKNOWN cards each showing 0 visits, no first/last seen timestamps, and no emotion data — confirming that the system makes no false positive identity assignments for unregistered persons. The HAPPY emotion label is visible on the video overlay for Akshat. Notably, the browser URL bar displays 10.187.95.217:5001/attendance, confirming that the Flask web dashboard was being accessed from a different device on the local network — demonstrating the system's viability as a network-accessible monitoring and attendance solution.

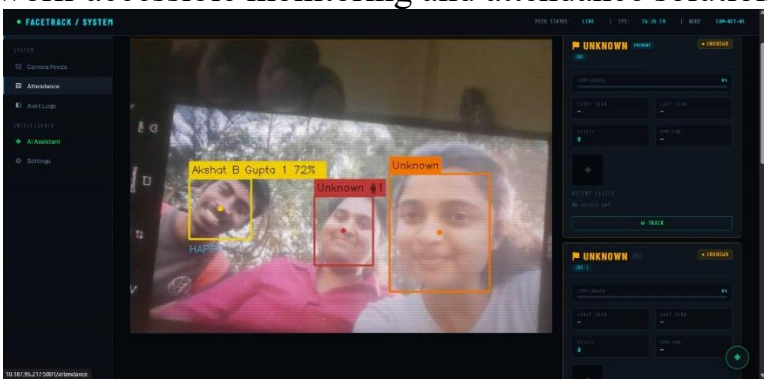


Figure 12

The most demanding test scenario captured during evaluation: four faces detected simultaneously in a single frame. Akshat B Gupta (66% confidence, yellow box) is correctly labelled as PRIMARY, while three background individuals are assigned Unknown #1, Unknown #2, and Unknown #3 (all red boxes). The system correctly prioritises the enrolled person as the primary tracked face and assigns distinct unknown track IDs to all remaining individuals without producing any false identity matches. The HAPPY emotion label is displayed for Akshat. The right-side panel records confidence at 55.6%, visit count 1, emotion happy, two previously captured snapshot thumbnails, and a 60-second recent visit entry. This test confirms correct primary-face prioritisation logic and robust multi-face track assignment in crowded real-world environments.

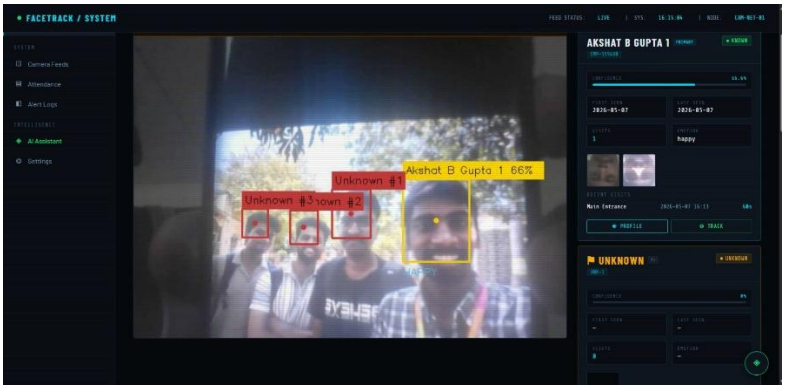


Figure 13

An earlier build of the system demonstrating the Camera Feed dashboard. Achala is

recognised at 80% confidence (yellow box) with FEAR emotion displayed on the overlay, alongside two unregistered persons (orange and red boxes). The right panel shows the Camera Select widget listing Webcam (dev 0) as ACTIVE and USB Cam A (dev 1) as available for one-click live switching. The Camera Info panel confirms: source Webcam (dev 0), resolution 640×480, model MediaPipe FD, confidence threshold ≥80%, and status RUNNING. The Recent Alerts panel logs a sequence of camera disconnection and reconnection events, verifying that the automatic reconnection watchdog and alert generation pipeline are both functional. This screenshot also confirms that the system correctly handles hardware interruptions without requiring a full restart.

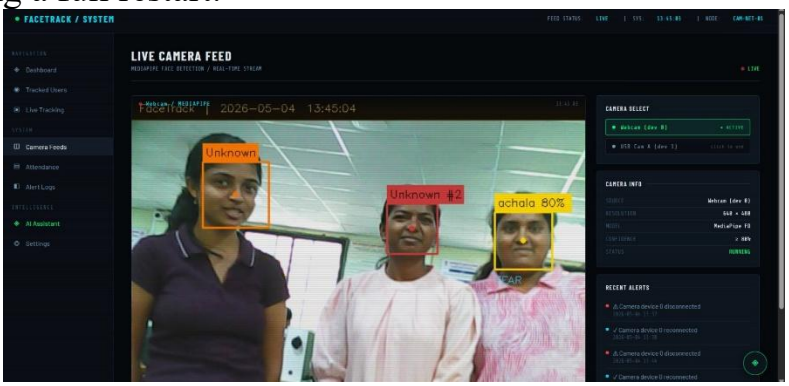


Figure 14

Conclusion

The FaceTrack project successfully demonstrates a real-time face detection, recognition, and servo-tracking system using a standard laptop and an Arduino Uno microcontroller. By replacing the earlier Raspberry Pi-based setup with direct USB serial communication, the system achieved lower latency, improved servo stability, reduced hardware complexity, and lower overall cost. The use of a proportional control algorithm with smoothing and deadzone filtering ensures stable and accurate face tracking even under noisy detection conditions. The Flask-based dashboard provides useful features such as live video streaming, attendance management, user enrolment, and AI-powered monitoring. The modular architecture also makes the system easy to maintain and extend for future applications in smart surveillance, automated attendance systems, and robotics. Overall, the project successfully achieves all its objectives and presents a reliable, scalable, and cost-effective integration of computer vision with electromechanical control.

Future Scope

The FaceTrack system can be further enhanced by replacing the SG90 servo with a stepper motor for smoother and full 360° rotation, as well as by adding multiple camera setups for wider coverage. Advanced AI models such as YOLOv8 can be integrated for improved multi-object tracking, while devices like NVIDIA Jetson Nano or Raspberry Pi 5 can enable standalone edge-AI deployment with higher processing performance. Additional features such as gesture-based control using

MediaPipe, voice command support using OpenAI Whisper, cloud integration through Firebase or AWS, and vibration stabilisation using IMU sensors can improve usability and reliability. In the future, reinforcement learning-based adaptive controllers may also be implemented to automatically optimise tracking performance without manual calibration.

References

1. R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2022.
2. P. Viola and M. Jones, "Rapid Object Detection using a Boosted Cascade of Simple Features," *IEEE CVPR*, 2001.
3. G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*, O'Reilly Media, 2008.
4. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," *IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
5. L. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," *IEEE CVPR Workshops*, 2019.
6. S. Monk, *Programming the Raspberry Pi: Getting Started with Python*, McGraw-Hill, 2019.
7. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
8. S. Patil and R. Kulkarni, "Servo-Based Face Tracking System Using Vision Algorithms," *IEEE International Conference on IoT and Intelligent Systems*, 2020.
9. A. Shinde and P. Kulkarni, "Smart Mirror Interface Using Interactive Vision Systems," *IEEE HCI Conference*, 2024.
10. OpenCV Documentation. Available: https://opencv.org
11. MediaPipe Documentation. Available: https://mediapipe.dev
12. Raspberry Pi Documentation. Available: https://www.raspberrypi.com/documentation
13. Google Cloud Speech-to-Text API. Available: https://cloud.google.com/speech-to-text
14. Python Software Foundation, Python Documentation. Available: https://www.python.org/doc
15. TowerPro, SG90 Micro Servo Motor Specifications. Available: https://www.towerpro.com.tw
16. Robocraze – Electronics Components and Servo Motors. Available: https://robocraze.com
17. FlyRobo – USB Camera Modules and Raspberry Pi Accessories. Available: https://www.flyrobo.in
18. Quartz Components – Raspberry Pi Boards and Modules. Available: https://www.quartzcomponents.com
19. Arduino, "Servo Motor Control Basics." Available: https://www.arduino.cc
20. GeeksforGeeks, Python and OpenCV Tutorials. Available:

https://www.geeksforgeeks.org
21. IEEE Xplore Digital Library. Available: https://ieeexplore.ieee.org
22. ResearchGate – Technical Papers Repository. Available: https://www.researchgate.net
23. MIT OpenCourseWare – Computer Vision Resources. Available: https://ocw.mit.edu
24. Towards Data Science – Vision & AI Articles. Available: https://towardsdatascience.com
25. Balakrishnan, B. & Raghavendra, R., *Embedded System Design*, Wiley India, 2018.